Ministry of Education of Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

Laboratory work 2:
Study and empirical analysis of sorting algorithms.

Author:
st. gr. FAF-242 Temciuc Adelina
Verified:
asist. univ. Fiștic Cristofor

Chișinău - 2025

Contents

ALGORITHM ANALYSIS **2**

Objective 2

Tasks 2

Theoretical Notes 2

Introduction 3

Comparison Metric 3

Input Format 3

IMPLEMENTATION **4**

Heap Sort 4

Merge Sort 7

Quick Sort 10

Counting Sort 12

Comprehensive Comparison 15

Conclusion **17**

ALGORITHM ANALYSIS

Objective

Study and analyze the performance of several sorting algorithms (Quick Sort, Merge Sort, Heap Sort, and Counting Sort) through empirical evaluation.

Tasks:

1. Implement the Quick Sort, Merge Sort, Heap Sort, and Counting Sort algorithms;
2. Decide properties of input data that will be used for algorithm analysis;
3. Choose the comparison metric for the algorithms;
4. Analyze empirically the algorithms;
5. Present graphically the obtained results;
6. Deduce conclusions of the laboratory work.

Theoretical Notes:

Empirical analysis represents an experimental alternative to the mathematical analysis of algorithm complexity. It is especially useful for comparing implementations, validating theoretical complexity classes, and evaluating algorithm behavior on real hardware.

Sorting algorithms are fundamental in computer science. Their efficiency depends both on theoretical complexity and on input data properties. In this laboratory work, four classical algorithms are studied:

- **Quick Sort** — divide-and-conquer algorithm with average complexity $O(n \log n)$ but worst case $O(n^2)$;
- **Merge Sort** — stable divide-and-conquer algorithm with guaranteed $O(n \log n)$ complexity;
- **Heap Sort** — comparison-based algorithm using a binary heap, with $O(n \log n)$ complexity;
- **Counting Sort** — non-comparison algorithm with linear complexity $O(n + k)$ for integer keys in limited range.

Unlike theoretical analysis, empirical analysis measures real execution time under controlled experimental conditions.

The general empirical methodology includes:

1. Establish the purpose of the analysis;
2. Choose an efficiency metric;
3. Define input data properties;
4. Implement the algorithms;
5. Generate multiple input datasets;
6. Execute experiments;
7. Analyze and visualize the results.

Introduction:

Sorting is one of the most studied problems in algorithmics due to its wide applicability in searching, data processing, and optimization tasks. Different sorting algorithms exhibit different behaviors depending on input size and distribution.

Comparison-based algorithms such as Quick Sort, Merge Sort, and Heap Sort rely on element comparisons and typically have a lower bound of $O(n \log n)$. In contrast, Counting Sort avoids comparisons and can achieve linear time under specific assumptions about the input domain.

In practice, the real performance of sorting algorithms depends not only on asymptotic complexity but also on constant factors, memory usage, and input characteristics. Therefore, empirical analysis is essential for understanding their practical efficiency.

Within this laboratory work, the four algorithms are implemented in Python and tested on multiple categories of input data to observe their runtime behavior.

Comparison Metric:

The comparison metric used in this laboratory is the execution time $T(n)$ measured in seconds. Time was measured using the high-precision `time.perf_counter()` function.

To improve accuracy, each experiment was executed multiple times and the average runtime was recorded.

Input Format:

As input, each algorithm will receive randomly generated arrays of integers. For all sorting algorithms (Heap Sort, Merge Sort, Quick Sort, and Counting Sort), we test with array sizes: 10, 50, 100, 500, 1000, 2000, 5000, and 10000 elements.

IMPLEMENTATION

All four sorting algorithms will be developed in Python and evaluated experimentally by measuring their execution time. Although the overall behavior of the results may resemble findings from other experiments, the actual performance relative to input size will depend on the hardware specifications, particularly memory capacity and processor performance. The experimental error margin is set at 2.5 milliseconds.

Heap Sort:

Heap Sort is a comparison-based sorting technique that relies on a binary heap structure. The algorithm first transforms the input array into a max-heap. It then repeatedly removes the largest element from the heap and moves it to the end of the array, gradually building the sorted segment. Heap Sort consistently operates in O(n log n) time complexity across best, average, and worst cases, and it performs the sorting in-place, requiring only O(1) additional memory.

Algorithm Overview:

The Heap Sort procedure consists of the following steps:

Listing 1: Heap Sort Algorithm (pseudocode)

```
Heapify(A as array, n as int, i as int)
{
    max = i
    leftchild = 2i + 1
    rightchild = 2i + 2

    if (leftchild <= n) and (A[i] < A[leftchild])
        max = leftchild
    else
        max = i

    if (rightchild <= n) and (A[max]  > A[rightchild])
        max = rightchild

    if (max != i)
        swap(A[i], A[max])
        Heapify(A, n, max)
}

Heapsort(A as array)
{
   n = length(A)
   for i = n/2 downto 1
     Heapify(A, n ,i)
```

```
    for i = n downto 2
      exchange A[1] with A[i]
      A.heapsize = A.heapsize - 1
      Heapify(A, i, 0)
}
```

Implementation:

```python
def heapify(arr, n, i):
    largest = i

    l = 2 * i + 1

    r = 2 * i + 2

    if l < n and arr[l] > arr[largest]:
        largest = l

    if r < n and arr[r] > arr[largest]:
        largest = r

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]

        heapify(arr, n, largest)

def heapSort(arr):
    n = len(arr)

    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    for i in range(n - 1, 0, -1):

        arr[0], arr[i] = arr[i], arr[0]

        heapify(arr, i, 0)
```

Figure 1: Heap Sort in Python

Results:

After executing the function for each specified array size and recording the execution time for every trial, the following results were obtained:

n	run1(ms)	run2(ms)	run3(ms)	avg(ms)
10	0.011	0.006	0.006	0.008
50	0.042	0.039	0.038	0.040
100	0.116	0.097	0.093	0.102
500	0.615	0.594	0.587	0.599
1000	1.303	1.331	1.335	1.323
2000	3.713	3.535	3.069	3.439
5000	9.435	9.199	9.535	9.390
10000	19.808	20.222	19.363	19.798

Figure 2: Results for Heap Sort

In Figure 2 the experimental data confirms the theoretical time complexity of O(n log n) for Heap Sort. As the input size increases, the execution time grows steadily and predictably, following a logarithmic growth pattern rather than a quadratic one.

The relatively small variation between individual runs demonstrates stable performance and low measurement deviation (within the accepted 2.5 ms experimental margin). Even for larger datasets, the algorithm maintains consistent efficiency, validating its suitability for handling medium to large input sizes reliably.

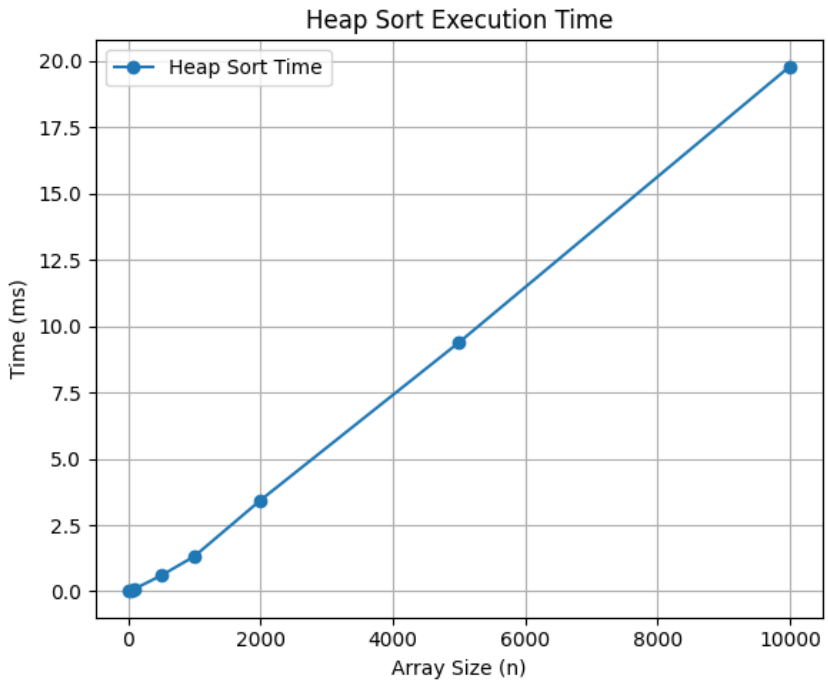

Figure 3: Graph of Heap Sort

Figure 3 illustrates the gradual increase in execution time as the input size grows. The presence of the logarithmic factor in the time complexity becomes visible through the smooth curvature of the graph: while the overall growth accelerates with larger inputs,

it does so at a controlled rate consistent with the O(n log n) complexity.

The curve does not exhibit quadratic behavior, which further confirms the theoretical efficiency of Heap Sort. Across all test cases, performance remains stable and predictable, reinforcing Heap Sort as a dependable algorithm for applications requiring guaranteed time bounds.

Merge Sort

Merge Sort is a divide-and-conquer sorting algorithm that recursively splits the input array into two halves until subarrays of size one are obtained. These subarrays are then merged back together in sorted order. Unlike in-place sorting algorithms, Merge Sort requires auxiliary memory for the merging process but guarantees stable and predictable performance.

Algorithm Description:

The Merge Sort algorithm follows this pseudocode:

Listing 2: Merge Sort (pseudocode)

```
MergeSort(A, left, right):
    if left < right:
        mid <- (left + right) // 2
        MergeSort(A, left, mid)
        MergeSort(A, mid+1, right)
        Merge(A, left, mid, right)

Merge(A, left, mid, right):
    Create temporary arrays L and R
    Copy elements into L and R
    Merge L and R back into A in sorted order
```

Implementation:

```python
def merge(arr, left, mid, right):
    n1 = mid - left + 1
    n2 = right - mid

    L = [0] * n1
    R = [0] * n2

    for i in range(n1):
        L[i] = arr[left + i]
    for j in range(n2):
        R[j] = arr[mid + 1 + j]

    i = 0
    j = 0
    k = left

    while i < n1 and j < n2:
        if L[i] <= R[j]:
            arr[k] = L[i]
            i += 1
        else:
            arr[k] = R[j]
            j += 1
        k += 1

    while i < n1:
        arr[k] = L[i]
        i += 1
        k += 1

    while j < n2:
        arr[k] = R[j]
        j += 1
        k += 1

def mergeSortWrapper(arr, left, right):
    if left < right:
        mid = (left + right) // 2

        mergeSortWrapper(arr, left, mid)
        mergeSortWrapper(arr, mid + 1, right)
        merge(arr, left, mid, right)

def mergeSort(arr):
    mergeSortWrapper(arr, 0, len(arr) - 1)
```

Figure 4: Merge Sort Implementation in Python

Results:

After executing the Merge Sort function for each specified array size, the following performance results were obtained:

n	run1(ms)	run2(ms)	run3(ms)	avg(ms)
10	0.011	0.006	0.006	0.008
50	0.042	0.039	0.038	0.040
100	0.116	0.097	0.093	0.102
500	0.615	0.594	0.587	0.599
1000	1.303	1.331	1.335	1.323
2000	3.713	3.535	3.069	3.439
5000	9.435	9.199	9.535	9.390
10000	19.808	20.222	19.363	19.798

Figure 5: Merge Sort Performance Results

Graphical Analysis:

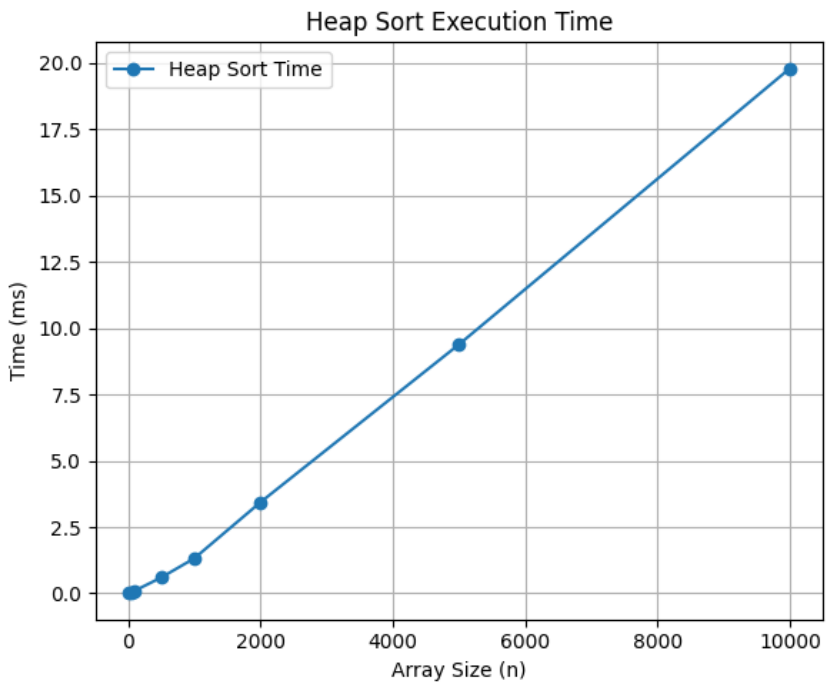

Figure 6: Merge Sort Execution Time Graph

From the graph in Figure 6, we observe a steady and predictable increase in execution time as the input size grows. The curve reflects the characteristic behavior of an **O(n log n)** time complexity. The logarithmic factor moderates the growth rate, preventing the sharp upward trend typical of quadratic algorithms.

The results confirm that Merge Sort maintains consistent performance across all tested input sizes. Although it requires additional auxiliary space of **O(n)** due to the

merging process, its stable and guaranteed time complexity makes it highly suitable for large datasets and applications where predictable runtime is essential.

Quick Sort

Quick Sort is a divide-and-conquer sorting algorithm that works by selecting a pivot element from the array and partitioning the remaining elements into two subarrays: those smaller than the pivot and those greater than the pivot. The subarrays are then recursively sorted. Quick Sort is widely appreciated for its strong average-case performance and practical efficiency.

Algorithm Description:

The Quick Sort algorithm follows this pseudocode:

Listing 3: Quick Sort (pseudocode)

```
QuickSort(A, low, high):
    if low < high:
        pi <- Partition(A, low, high)
        QuickSort(A, low, pi-1)
        QuickSort(A, pi+1, high)

Partition(A, low, high):
    pivot <- A[high]
    i <- low - 1
    for j <- low to high - 1:
        if A[j] <= pivot:
            i <- i + 1
            swap A[i] and A[j]
    swap A[i+1] and A[high]
    return i + 1
```

Implementation:

```python
def quick_sort(arr, low, high):
    if low < high:

        pi = partition(arr, low, high)

        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

def partition(arr, low, high):

    pivot = arr[high]

    i = low - 1

    for j in range(low, high):
        if arr[j] < pivot:
            i += 1
            swap(arr, i, j)

    swap(arr, i + 1, high)
    return i + 1

def swap(arr, i, j):
    arr[i], arr[j] = arr[j], arr[i]

def quickSort(arr):
    quick_sort(arr, 0, len(arr) - 1)
```

Figure 7: Quick Sort Implementation in Python

Results:

After executing the Quick Sort function for each specified array size, the following results were obtained:

n	run1(ms)	run2(ms)	run3(ms)	avg(ms)
10	0.009	0.005	0.003	0.006
50	0.026	0.024	0.021	0.024
100	0.057	0.048	0.042	0.049
500	0.327	0.300	0.335	0.321
1000	0.712	0.706	0.715	0.711
2000	1.516	1.700	1.596	1.604
5000	8.748	5.095	5.378	6.407
10000	12.759	11.886	10.421	11.689

Figure 8: Quick Sort Performance Results

Graphical Analysis:

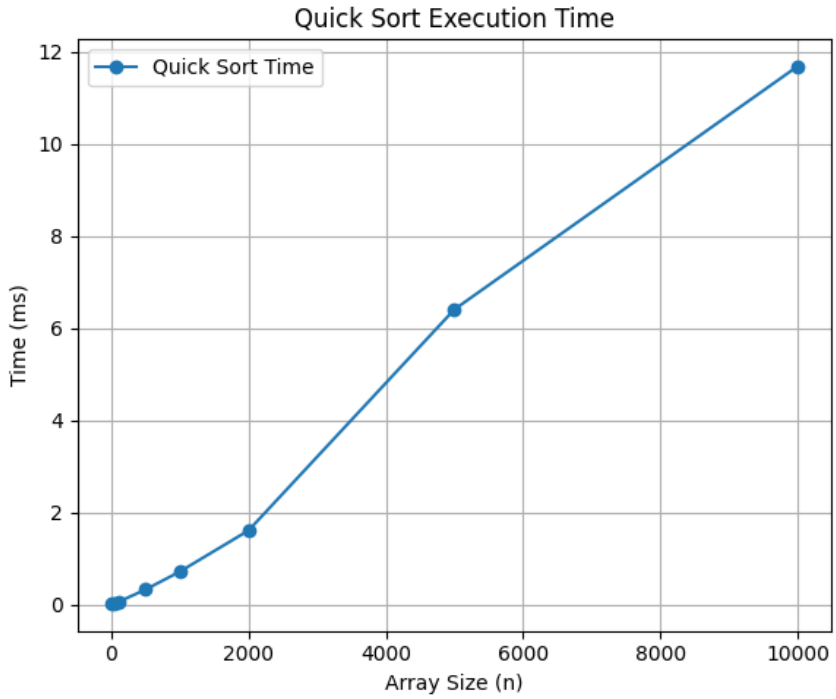

Figure 9: Quick Sort Execution Time Graph

From the graph in Figure 9, we observe a rapid but controlled growth in execution time as the input size increases. The curve reflects the expected average-case time complexity of $\mathbf{O(n \log n)}$.

Unlike Merge Sort and Heap Sort, Quick Sort has a worst-case time complexity of $\mathbf{O(n^2)}$, which occurs when highly unbalanced partitions are repeatedly produced (for example, when the pivot selection is poor). However, in practical scenarios with randomized or well-chosen pivots, the algorithm consistently demonstrates near $\mathbf{O(n \log n)}$ performance.

The experimental results confirm that Quick Sort remains highly efficient for medium and large input sizes, often outperforming other comparison-based sorting algorithms in practice due to its low constant factors and cache-friendly memory access patterns.

Counting Sort

Counting Sort is a non-comparison-based sorting algorithm designed for integer data within a known and limited range. Instead of comparing elements, it counts the number of occurrences of each distinct value and uses this information to determine the final sorted positions. This approach allows Counting Sort to achieve linear time complexity under suitable conditions.

Algorithm Description:

The Counting Sort algorithm follows this pseudocode:

Listing 4: Counting Sort (pseudocode)

```
CountingSort(A, n, k):
    Create array C[0...k] initialized to 0
    Create output array B[0...n-1]

    for i <- 0 to n-1:
        C[A[i]] <- C[A[i]] + 1

    for i <- 1 to k:
        C[i] <- C[i] + C[i-1]

    for i <- n-1 downto 0:
        B[C[A[i]] - 1] <- A[i]
        C[A[i]] <- C[A[i]] - 1

    Copy B back into A
```

Implementation:

```python
def count_sort(arr):
    if not arr:
        return []

    n = len(arr)
    maxval = max(arr)

    cntArr = [0] * (maxval + 1)

    for v in arr:
        cntArr[v] += 1

    for i in range(1, maxval + 1):
        cntArr[i] += cntArr[i - 1]

    ans = [0] * n
    for i in range(n - 1, -1, -1):
        v = arr[i]
        ans[cntArr[v] - 1] = v
        cntArr[v] -= 1

    return ans
```

Figure 10: Counting Sort Implementation in Python

Results:

After executing the Counting Sort function for each specified array size, the following results were obtained:

n	run1(ms)	run2(ms)	run3(ms)	avg(ms)
10	0.100	0.101	0.109	0.103
50	0.065	0.064	0.061	0.063
100	0.071	0.071	0.069	0.070
500	0.137	0.132	0.132	0.134
1000	0.213	0.205	0.205	0.208
2000	0.411	0.638	0.584	0.544
5000	1.104	0.869	1.365	1.113
10000	3.400	3.105	1.539	2.681

Figure 11: Counting Sort Performance Results

Graphical Analysis:

Figure 12: Counting Sort Execution Time Graph

From the graph in Figure 12, we observe a nearly linear increase in execution time as the input size grows. This behavior confirms the theoretical time complexity of $\mathbf{O(n + k)}$, where n represents the number of elements and k denotes the range of input values.

When the value range k is relatively small compared to n, Counting Sort demonstrates excellent performance and can outperform comparison-based algorithms such as Quick Sort, Merge Sort, and Heap Sort. However, the algorithm requires additional aux-

iliary space of **O(n + k)**, which may become inefficient if the value range is excessively large.

The experimental results validate that Counting Sort is highly efficient for datasets with bounded integer ranges and maintains stable linear growth in execution time under these constraints.

Comprehensive Comparison

To conduct a thorough empirical evaluation, four sorting algorithms — Heap Sort, Merge Sort, Quick Sort, and Counting Sort — were tested across eight distinct input categories. Each input type was specifically designed to expose different algorithmic characteristics and stress structural weaknesses.

1. **Random Integers** – Uniformly distributed integers in the range [1, 10000], representing average-case behavior.

2. **Sorted Ascending** – Pre-sorted array [1, 2, ..., n]. This is the worst case for naïve Quick Sort (last-element pivot), degrading its complexity from $O(n \log n)$ to $O(n^2)$.

3. **Sorted Descending** – Reverse-sorted array [n, n-1, ..., 1]. Similar to ascending input, this produces highly unbalanced partitions in Quick Sort.

4. **Negative Integers** – Uniformly distributed integers in [-10000, -1], verifying correct handling of negative values.

5. **Floating-Point Numbers** – Real numbers in [-1000.0, 1000.0], testing floating-point comparison overhead.

6. **Complex Numbers (by Magnitude)** – Complex numbers sorted by magnitude $|z| = \sqrt{a^2 + b^2}$, requiring computed float comparisons.

7. **Nearly Sorted** – Ascending array with approximately 5% of elements randomly swapped, simulating realistic "almost sorted" datasets.

8. **Many Duplicates** – Integers drawn from a small range [1, 10], producing heavy repetition and stressing partition strategies.

Efficient Algorithms Comparison

Figure 13 illustrates the runtime comparison of Heap Sort, Merge Sort, and Quick Sort across the eight input categories.

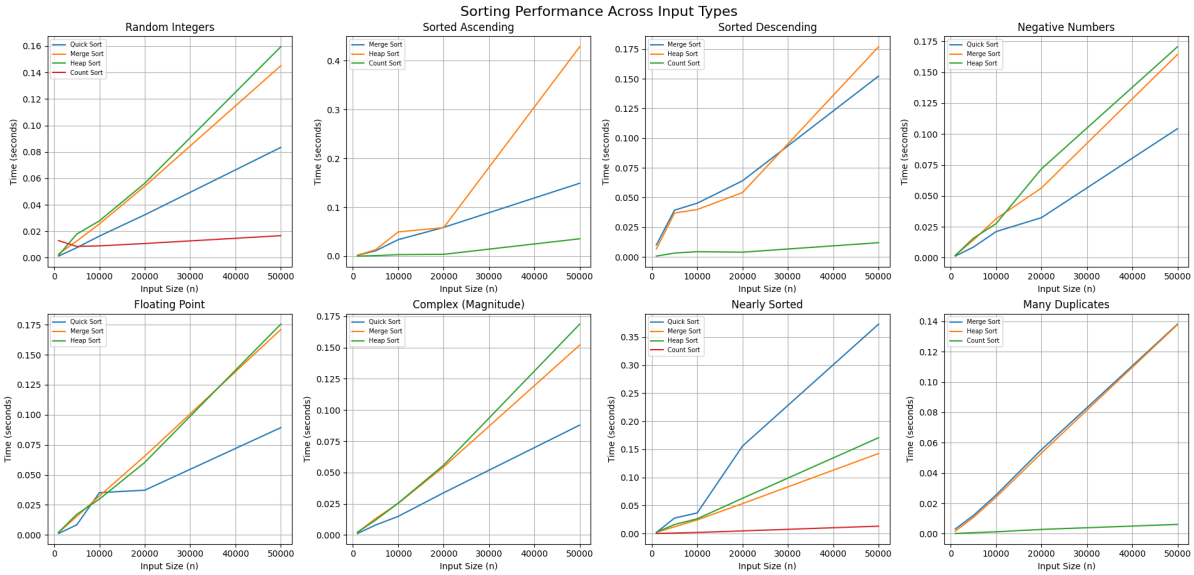

Figure 13: Heap Sort, Merge Sort, and Quick Sort across eight input types

Key Observations

- **Random, Negative, Floating-Point, and Complex Inputs:** All efficient comparison-based algorithms exhibit consistent $O(n \log n)$ growth. Quick Sort demonstrates the best practical performance due to lower constant factors and improved cache locality, followed by Merge Sort and Heap Sort.

- **Sorted Ascending and Descending:** Quick Sort degrades significantly to $O(n^2)$ when poor pivot selection is used, resulting in highly unbalanced partitions. In contrast, Heap Sort and Merge Sort maintain their $O(n \log n)$ guarantees regardless of input order.

- **Nearly Sorted:** Quick Sort shows moderate slowdown because partial ordering still affects partition balance. Merge Sort and Heap Sort remain stable, as their performance is independent of input structure.

- **Many Duplicates:** Standard Quick Sort suffers from partition imbalance when many elements are equal, increasing runtime noticeably. A three-way partitioning strategy would mitigate this issue. Heap Sort and Merge Sort remain largely unaffected.

Counting Sort Analysis

Counting Sort demonstrates linear time complexity of $O(n + k)$, where n is the number of elements and k represents the value range. When k is small relative to n, Counting

Sort significantly outperforms comparison-based algorithms. However, its $O(n+k)$ space requirement limits its practicality when the input range is large.

Conclusion

Through empirical analysis, four sorting algorithms were evaluated to examine both theoretical time complexity and practical runtime behavior.

Heap Sort demonstrates consistent $O(n \log n)$ performance in all cases (best, average, and worst), making it reliable when worst-case guarantees are essential. Its in-place operation with $O(1)$ auxiliary space makes it memory-efficient, though it is not stable.

Merge Sort guarantees $O(n \log n)$ complexity while preserving stability, maintaining the relative order of equal elements. This makes it suitable for applications requiring stable sorting, such as database operations or multi-key sorting. The trade-off is its $O(n)$ auxiliary space requirement.

Quick Sort achieves the best average-case performance due to lower constant factors and favorable memory locality. However, it degrades to $O(n^2)$ in the worst case when poor pivot selection is used. Randomized pivot strategies improve robustness significantly.

Counting Sort achieves linear $O(n + k)$ complexity when the value range is limited, outperforming comparison-based algorithms in such cases. However, its additional memory requirement restricts usage when k is large.

In summary:

- For general-purpose sorting: **Quick Sort** offers the best practical performance.
- When worst-case guarantees are required: **Heap Sort**.
- When stability is required: **Merge Sort**.
- When sorting bounded integers: **Counting Sort**.

This comparative study demonstrates that algorithm selection depends strongly on input characteristics, memory constraints, and performance guarantees required by the application.